Lab 10 — Questions

Question 1 [Diagnosis]

A staging server has run out of disk space. `podman system df` looks clean: few images, no orphaned volume. Yet `du -sh ~/.local/share/containers/storage/overlay-containers/*/userdata/` shows one container at 40 GB. What is filling the disk, why does `system df` not show it in the "reclaimable" column, and what are the two fixes (immediate and lasting)?

Question 2 [Prediction]

```
podman run -d --name api --memory 256m -e JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=90" api:1.0
```

The API works for ten minutes, then the container disappears from `podman ps`; `podman ps -a` shows it as `Exited (137)`. The application logs contain no `OutOfMemoryError`. Explain the exact sequence, say which command confirms the diagnosis, and propose two fixes.

Question 3 [Understanding]

What do `podman stats` and `podman top` respectively show for a container, how do they differ from `podman inspect`, and why can `podman stats` display a **limit** of 32 GB for a container started without `--memory` on a 32 GB machine?

Question 4 [Diagnosis]

```
FROM localhost/api:1.0
HEALTHCHECK --interval=5s --retries=3 CMD wget -qO- http://localhost:8080/actuator/health ||
exit 1
```

```
$ podman build -t api:1.1 .
$ podman run -d --name api api:1.1
$ podman healthcheck run api
Error: container … has no defined healthcheck
```

The Dockerfile is correct. Why does Podman not see the healthcheck, how do you check it on the image, and which option fixes the build? What would have happened with Docker?

Question 5 [Analysis]

An `api` container turns `unhealthy` after a database connection leak. With rootless Podman alone (no Compose, no Kubernetes), what happens? What does Docker do in the same situation? Which Podman option allows an automatic reaction, and why does an orchestrator remain the real answer?

Question 6 [Prediction]

```
podman run --rm --read-only --cap-drop ALL --user 1000 --security-opt no-new-privileges --
entrypoint sh api:1.0 -c 'id; touch /app/x; touch /tmp/x; java -version'
```

Predict the output line by line, then say what this command is missing for a real Spring Boot API to start (hint: Tomcat and the JVM write somewhere).

Question 7 [Understanding]

Explain why `chown nobody /etc/hostname` succeeds in a normally started `alpine` container but fails with `Operation not permitted` under `--cap-drop ALL`, when in both cases the process is `root` (UID 0) inside the container. What does this example reveal about the difference between "being root" and "having privileges"?

Question 8 [Analysis]

A team's CI pushes every build as `api:latest` and the servers run `podman pull api:latest` then restart. Name three incidents this strategy makes possible, then describe the recommended tag strategy and what deploying **by digest** guarantees beyond deploying by version tag.

Question 9 [Diagnosis]

```
$ podman push localhost:5000/api:1.0
Error: … pinging container registry localhost:5000: Get "https://localhost:5000/v2/": http:
server gave HTTP response to HTTPS client
```

Explain the cause, give the fix for a local test and the one for a company registry, and say why the former must never become the latter.

Question 10 [Analysis]

After `podman push --tls-verify=false localhost:5000/api:1.0`, then `podman rmi localhost:5000/api:1.0` and `podman pull --tls-verify=false localhost:5000/api:1.0`, `podman images` shows `localhost/api:1.0` and `localhost:5000/api:1.0` with the **same** `IMAGE ID`. Explain what the registry actually received and returned, why the ID is identical, and what guarantee `podman image inspect --format '{{.Digest}}'` adds.

Question 11 [Diagnosis]

A colleague runs `podman system prune -a --volumes -f` on a CI agent "to free some space" while the test stack is running. List what is removed and what is not, the consequences for the next build, and the sequence they should have run.

Question 12 [Analysis]

Here is a Quadlet:

```
[Container]
Image=localhost/api:1.0
PublishPort=18080:8080
EnvironmentFile=%h/labo-docker/09/api.env
[Install]
WantedBy=default.target
```

Explain what `systemctl --user daemon-reload` and `systemctl --user enable --now api` produce, why this service survives `wsl --shutdown` where `--restart=always` fails, and what still has to be configured on WSL for the service to start without opening a session.

Question 13 [Analysis]

Describe a complete CI/CD pipeline for `api`, from `git push` to production, in five steps, stating for each: the command or tool, what blocks the pipeline on failure, and which lab of this series

introduced the notion. Where do the image scan, the `HEALTHCHECK`, the resource limits and the digest fit?